

Handling Network Faults in Distributed AI Training: Failover is Now an Option

Xin Zhe Khooi^{*‡}, Zhuo Jiang[†], Pan Xie, Zhigang Cui, Meng Wang, Yuze Jin[‡], Pengfei Huo, Dongyang Wang, Lulu Chen, Lei Wang, Liaoyuan Feng, Xiaodong Liu, Peng Li, Qinlong Wang, Yang Bai, Yongcan Wang, Hao Jin, Jinshuai Sun, Shan Lu, Xiang Shi, Yingkai Zhao, Haiquan Chen, Yi Li, Jianxi Ye, Mun Choon Chan[‡]

[‡]National University of Singapore ByteDance

Abstract

Distributed AI training often suffers from network faults. Network faults, especially at the last hop between a switch and a host, result in loss of connectivity, resulting in training job stalls and eventual failure. This is typically managed through a fail-stop mechanism, followed by a restart, incurring significant inefficiencies. We present ReCCL, the first network fault-tolerant collective communication library (CCL) that allows training progress to be preserved by seamlessly failing over to alternate paths when a network fault occurs. During failover, ReCCL keeps communication states synchronized while using dynamic channel load balancing and intra-host GPU routing to improve communication performance. Our evaluations demonstrate that ReCCL can perform failover seamlessly with minimal performance losses. Additionally, our simulations also demonstrate that failover can be effectively used to achieve significant savings in GPU hours for large-scale distributed AI training workloads.

CCS Concepts: • Networks → Network reliability; Data center networks.

Keywords: distributed AI training, collective communication libraries, NCCL, failover, restart, RDMA

ACM Reference Format:

Xin Zhe Khooi, Zhuo Jiang, Pan Xie, Zhigang Cui, Meng Wang, Yuze Jin, Pengfei Huo, Dongyang Wang, Lulu Chen, Lei Wang, Liaoyuan Feng, Xiaodong Liu, Peng Li, Qinlong Wang, Yang Bai, Yongcan Wang, Hao Jin, Jinshuai Sun, Shan Lu, Xiang Shi, Yingkai Zhao, Haiquan Chen, Yi Li, Jianxi Ye, and Mun Choon Chan. 2026. Handling Network Faults in Distributed AI Training: Failover is Now an Option. In *21st European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland UK. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3767295.3769322>

^{*}This work was done while interning at ByteDance.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EUROSYS '26, Edinburgh, Scotland UK

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04

<https://doi.org/10.1145/3767295.3769322>

1 Introduction

Distributed AI training is highly sensitive to failures [9, 41]. Training job failures happen due to a variety of causes, including software crashes, memory errors, GPU failures, or network faults [9, 16, 28].

In this paper, we focus on the handling of network faults affecting the last hop between a switch and a host, which causes worker nodes to lose connectivity, resulting in training job stalls and eventual failure. These failures include events such as link flapping, RDMA NICs (RNIC) failures, and top-of-the-rack (ToR) switch outages. In our experience, such failures constitute the root cause of more than 12 % of documented failed jobs. Several other hyperscalers have also reported failure rates of similar magnitude [9, 16, 28] as well.

Current approaches to fault handling necessitate a fail-stop response where tasks are subsequently rescheduled on healthy nodes and restarted from the last checkpoint [16, 20, 32, 57], *regardless of the failure cause*. This has resulted in the loss of training progress since the prior checkpoint, incurring significant overhead. As training scales to tens of thousands of GPUs and spans durations of weeks or months, handling faults more effectively is essential to maintaining the efficiency of modern AI workloads.

Unlike errors caused by software, memory, or GPU, network faults are different in that they do not cause the underlying system to malfunction or training states to be lost. Thus, we argue that network faults should be managed differently. In particular, given the available redundancy through multiple RNICs on today's GPU servers, instead of indiscriminately restarting the job, with possibly a lower number of training nodes (i.e., scale down [24, 46]), another option is to *failover*. With failover, a job can proceed *in-place* using the same set of training nodes, with a slight degradation in communication performance by rerouting communication through alternative intra- and inter-host paths.

Given the potential benefits of using failover over restart to handle network faults, there is, however, no support for failover in the collective communication libraries (CCL) used with today's distributed AI training frameworks. For example, NVIDIA's NCCL [39], which many depend on today, assumes fault-free communication paths and does not handle network faults. Whenever the CCL loses connectivity, the

CCL stalls and eventually crashes. This causes the training job to have to be restarted.

To improve the resiliency of the CCL against network faults, there have been two lines of approach. The first approach is through hardware-level redundancy like hardware bonding [26, 35, 41] through using dual-port RNICs with redundant uplinks. However, such an approach is hardware- and topology-dependent. Furthermore, the training job will still have to restart if the RNIC fails. The second approach uses intermediate software layers to mask network faults from the CCL. For example, Lin et al. [25] proposed an application-agnostic userspace IB Verb provider offering RDMA failover capabilities. However, this requires extra software agents, increasing deployment complexity and creating an undesirable single point of failure. Additionally, being application-agnostic, this approach lacks visibility and control over intra-host paths, potentially resulting in sub-optimal GPU-to-GPU communication performance.

Given the shortcomings of the aforementioned approaches, we propose ReCCL, the first network-fault-tolerant CCL, supporting both seamless failover and recovery. We address the shortcomings of existing approaches in the following way.

ReCCL is topology-independent and can handle all communication failures between a switch and a host using failover. Next, ReCCL operates as an in-place replacement of NCCL with no need for intermediate software layers and/or additional software agents. Finally, ReCCL allows control over both the intra- and inter-host GPU-to-GPU communication paths, providing opportunities for ReCCL to maximize communication performance during failover.

Designing ReCCL, however, is non-trivial. There are two key challenges. First, as evidenced by the comments from NCCL’s author [1], introducing a primary-backup failover mechanism to address network faults within the CCL is a hard problem. For instance, the failover mechanism requires careful synchronization of the communication states between sender and receiver, and considerations on how the backup path resources should be provisioned. Second, failover inherently introduces communication performance degradation due to increased resource sharing and rerouting overhead. Thus, challenges lie in developing mechanisms that effectively minimize performance degradation, ensuring minimal disruption to the overall training efficiency.

We address the aforementioned two challenges through the following. First, to support seamless failover, we introduce abstract redundant RNICs at the CCL along with a primary-backup failover and recovery protocol, ensuring sender and receiver communication states remain synchronized during failover and recovery when the fault is rectified.

Second, to minimize communication performance losses during failover, we fully utilize the multiple RNICs on each node and adapt to the complex intra-host topologies. To that end, we present two CCL optimizations to improve communication performance during network faults. First, we

introduce a dynamic load balancing scheme within the CCL that redistributes data to minimize communication channel performance degradation. Second, we make use of the high-speed GPU interconnects to reroute inter-host traffic via an intermediate GPU to overcome the intra-host PCIe bottlenecks between a GPU-backup RNIC pair. We also carefully align ReCCL to take advantage of modern AI cluster network topologies – rail-only or rail-optimized [13, 41, 51]. For instance, we establish disjoint backup paths following the rails. Thus, even under a worst-case scenario with the ToR switch going down completely, training can proceed uninterrupted.

Our contributions. In this paper, we make the following contributions:

- We highlight the potential gain of using failover to handle network faults as compared to job restarts in distributed AI training workloads.
- ReCCL presents the design of the first CCL that incorporates network fault-tolerance, supporting both failover and recovery.
- ReCCL incorporates novel techniques to optimize CCL performance during network faults to address network bottlenecks after failover.
- Our implementation of ReCCL maintains compatibility with NVIDIA’s NCCL, allowing it to be used as an *in-place* replacement without recompilation of existing training frameworks.
- We conduct evaluations on ReCCL on our testbed to showcase its ability to seamlessly failover and perform recovery, with minimal training performance loss.

2 Motivation and Background

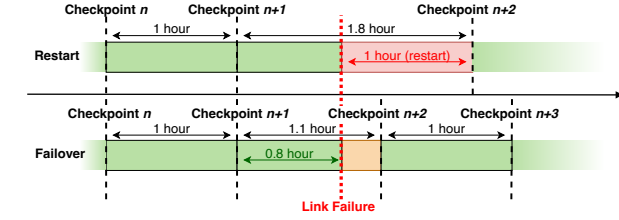
2.1 Job scheduler: Fault handling

Understanding failure handling requires examining the fleet management system, which consists of: (1) a job scheduler (a.k.a. central coordinator) managing job allocation and system health, and (2) per-worker node agents monitoring resource utilization and process events (e.g., training logs).

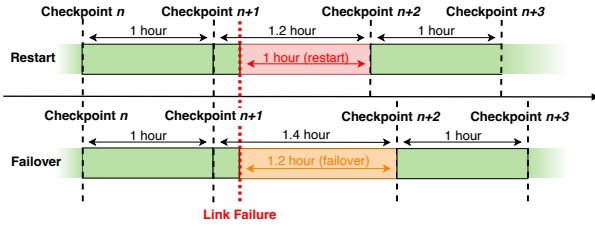
The job scheduler gathers real-time status from the agents to perform health checks and respond accordingly [20, 57]. When a worker node fails, training stalls, times out, and an agent notifies the scheduler. The system performs diagnostics and marks the node as “unhealthy”, ensuring it is excluded from rescheduling when the job restarts [16, 20, 57].

Reacting to network faults. Network faults between a host and a switch disrupt CCL communication, prompting RNIC retransmissions, which ultimately fail if connectivity is lost. This triggers a Completion Queue Event (CQE) error, halting CCL operations. The training framework, unaware of the failure, stalls until a watchdog process times out, leading to job termination and rescheduling.

Reaction time depends on the RNIC and underlying watchdog timeouts, and this varies from operator to operator. For example, He et al. [16] highlighted that it can be up to 30



(a) Failover is preferable: After checkpoint $n + 1$, a network fault occurs 0.8 hours into training. Restarting wastes this progress and adds 1 hour, while failover (at 50% slowdown) incurs an additional 0.1 hours to the remaining training time (0.2 hours) until the next checkpoint.



(b) Restart is preferable: After checkpoint $n + 1$, a network fault occurs 0.2 hours into training. Restarting adds 1 hour, while failover (at 50% slowdown) adds 0.4 hours to the remaining training time (0.8 hours) until the next checkpoint.

Figure 1. Motivating example.

minutes. A more sophisticated health-check system could detect CCL failures earlier and trigger faster restarts, reducing downtime to minutes [20].

2.1.1 The case for failover. Consider a distributed AI training job that performs regular checkpointing. When a network fault occurs, we consider two options: (1) restart immediately from the last checkpoint using healthy nodes, or (2) continue training (with potentially degraded performance) and restart on healthy nodes later at the next checkpoint.

For simplicity, assume an idealized scenario where there are no overheads involved in detecting the network fault, triggering a restart, and reloading the training job as shown in Fig. 1. Consider the scenario in which a network fault occurs not far from where the next checkpoint is to be performed (Fig. 1a). Restarting the job would waste all the processing done since the last checkpoint. It is evidently better if the job can continue by performing a failover to alternate communication paths without a restart. However, if the fault occurs shortly after a checkpoint has been performed (Fig. 1b), then restarting may be a better option.

When to failover. To make the discussion concrete, we present a model for the tradeoff seen by the job scheduler.

Let the checkpoint interval be the constant $T_{interval}$. When a network fault happens, let the time elapsed since the previous checkpoint be $t_{elapsed}$. The time remaining until the next checkpoint is simply $T_{interval} - t_{elapsed}$.

Next, let the RNIC retransmission timeout and the training framework/watchdog timeouts be $T_{timeout}$ and T_{detect} , respectively. Finally, let the overhead to restart the training job be T_{reload} . We assume a single-failure scenario.

Let $t_{restart}$ be the total time needed to reach the next checkpoint after a network fault using restart:

$$t_{restart} = T_{timeout} + T_{detect} + T_{reload} + T_{interval} \quad (1)$$

Let $t_{failover}$ be the total time needed to reach the next checkpoint after a network fault using failover. As the job will eventually be rescheduled on healthy nodes to restore performance after reaching the next checkpoint, we also factor in T_{reload} as part of the overhead:

$$t_{failover} = T_{timeout} + k(T_{interval} - t_{elapsed}) + T_{reload} \quad (2)$$

where $k(\geq 1)$ is the normalized increase in time incurred using failover over the baseline without network faults.

For the failover approach to be beneficial, the total time/overhead using failover should be less; therefore:

$$t_{failover} < t_{restart} \quad (3)$$

By combining the equations, we can express the condition for choosing failover over restart as:

$$t_{elapsed} > \frac{T_{interval}(k - 1) - T_{detect}}{k} \quad (4)$$

Thus, if $t_{elapsed}$ is sufficiently large, the amount of computation retained is sufficient to compensate for the slowdown introduced by failover. On the other hand, if $t_{elapsed}$ is small, a restart may be better since the amount of slowdown introduced by failover is more costly than the amount of loss computation.

Consider the case where t_{detect} is 0 and k is 1.5:

$$t_{elapsed} > \frac{T_{interval}}{3} \quad (5)$$

In this case, as long as $t_{elapsed}$ goes beyond $\frac{1}{3}$ of the checkpoint interval, failover is better than restart. In addition, we would like to highlight the following:

- In practice, t_{detect} is clearly not 0 and k is much smaller than 1.5 (see §4). These factors increase the operating region for choosing failover over restart.
- If the set of computing nodes used after restart is reduced, which is the case of *scaling down* [24, 46], the benefit of failover can be much larger (see §4.1).
- Failover is beneficial independent of the checkpointing interval, though the actual gain will reduce with smaller $T_{interval}$ (see §4.1).

2.2 Preliminaries on GPU-to-GPU collective communications with CCLs

Many widely used collective communication libraries (CCLs), such as ACCL [2, 10], RCCL [4, 5], NCCLX [28], TCCL [23], and MSCCL [31], are extensions of NVIDIA's open-source NCCL (NVIDIA Collective Communication Library) [39, 40].

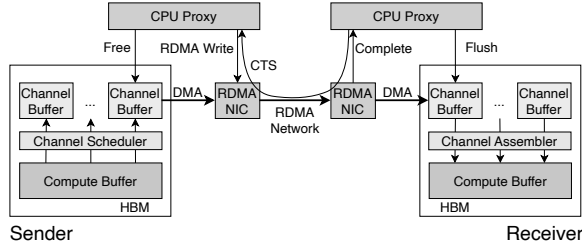


Figure 2. Overview of GPU-GPU communication architecture in CCLs, adapted from [13, Fig. 14].

Thus, our discussion focuses on NCCL, with insights applicable to many custom CCL implementations.

CCLs simplify GPU-to-GPU communication by providing high-level APIs to distributed AI training frameworks, enabling efficient data exchange for collective operations such as AllReduce, AllGather, ReduceScatter, and AlltoAll.

Communication groups. A communication group consists of a set of GPUs participating in a collective operation. This is often decided by the training framework, e.g., Megatron-LM, based on the training parallelism [53]. Each group may span multiple nodes, with communication optimized based on the group size and GPU connectivity.

Logical topologies. Based on the communication group, CCLs map operations to logical topologies, including: (1) ring: pass data among nodes in a circular fashion, and (2) tree: GPUs use hierarchical relationships to transfer data from one another. Topology selection depends on factors like the number of GPUs, message size, and operation type.

Communication channels. To maximize throughput, CCLs establish multiple parallel communication channels across a set of GPUs within the same communication group. These channels distribute the workload across available RNICs on the GPU system to fully utilize bandwidth resources. For each channel, all GPUs within the communication group are involved, but they may have different logical topologies or orderings based on proximity to the ingress and egress RNICs (Fig. 7a). In §3.2.1, we show how we adapted the existing channel workload distribution mechanism by modifying the channel scheduler to improve communication performance during network faults.

Synchronicity. For each channel, the total number of message passes per GPU must equal the number of collective operations performed, ensuring synchronization. This invariant is leveraged in our optimizations discussed in §3.2.1.

2.2.1 Inter-host GPU communications. Modern AI clusters interconnect GPU hosts using InfiniBand or RoCEv2 networks, with data transfer primarily handled via Remote Direct Memory Access (RDMA) through RNICs. Fig. 2 illustrates the inter-host GPU communication architecture.

CCLs typically adopt a receiver-driven model [13, Sec. 5.2.2], where the receiver signals readiness by sending a Clear-to-Send (CTS) message containing: (1) size and (2)

memory location. In practice, multiple CTSs can be issued in parallel for different memory locations. Upon receiving the CTS, the sender transmits data to the specified location. Both sender and receiver CPUs poll the RNIC’s Completion Queue (CQ) to confirm successful transfers via Completion Queue Events (CQEs) [22]. Upon completion, the data pointers advance, and the receiver issues the next CTS. Maintaining pointer synchronization is crucial for consistent operation.

Handling network faults. During network faults, RNICs generate error CQEs, causing CCL operations to halt, leading to training job restarts. ReCCL improves fault tolerance by seamlessly switching to backup paths (see later in §3.1) and includes mechanisms to detect and correct data pointer misalignments, ensuring correct operations throughout.

2.3 Limitation of existing network-fault tolerant approaches for distributed AI training

We now discuss how existing network resilience approaches are used/proposed for distributed AI training. At the same time, we highlight their drawbacks.

Redundant links and multi-homing. To improve the resiliency against network faults between a switch and a host, a common approach is to depend on hardware redundancy by performing hardware bonding [26, 35, 41] using dual-port RNICs with redundant uplinks. Qian et al. [41] has widely used this in deployment and proposed a new AI cluster network topology. In particular, Qian et al. [41] heavily depends on dual-port RNICs associated with each GPU. Through the use of dual-port RNICs, each GPU has two redundant links connected to different ToR switches located in different network planes, making it resistant to ToR switch failures.

While such an approach is effective, it suffers from one key drawback: it does not handle RNIC failures, which can happen quite often between a switch and a host [9, Table 1]. Additionally, given the dependence on dual-port RNICs and a specific network topology design, this further limits the applicability of this approach for existing clusters.

Even if the proposed setup by Qian et al. [41] is adopted for new clusters, Qian et al. [41] also acknowledges that maintaining and operating such a topology is costly, complex, and error-prone, especially in terms of cabling and network configuration. This makes it even less of an appeal when operators are looking to simplify the already-complex network operations for large-scale AI clusters.

Masking failures with software. Hardware aside, an alternative approach is to use intermediate software layers to mask failures from the upper-layer application. Recent work by Lin et al. [25] proposes LubeRDMA, an application-agnostic userspace IB Verbs library that masks the underlying network faults and handles them through a primary-backup scheme. Such an approach dynamically overrides the default IB Verbs API to perform network fault handling and thus masks failure from the upper-layer application, in this

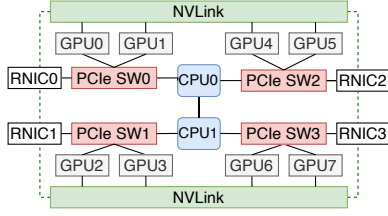


Figure 3. Example GPU system with 8 GPUs, 4 RNICs, and inter-GPU NVLink. Using GPU0 as a reference: if GPU0 uses RNIC0, the maximum achievable throughput is ~ 390 Gbps, otherwise it would be ~ 140 Gbps (64.1% loss). The throughput is measured using the RDMA perfest [3] tool.

case, the CCL. Additionally, this approach is independent of the underlying network topology and RNIC setup, e.g., single-port RNICs.

However, LubeRDMA [25] only supports two-sided Verbs, but not one-sided Verbs, which are used by CCLs for inter-host communications, making it inapplicable for our distributed AI training workloads. Even if one-sided Verbs can be supported¹, LubeRDMA requires additional external software agents across the entire cluster for inter-host coordination, which increases the deployment complexity and creates an additional single point-of-failure, which we want to avoid.

Even if adopted, during failover, such an approach may suffer from poor communication performance, where training performance can be significantly impacted, which makes failover a less appealing option over restarting, given the excessive performance slowdown. As an example, for the system in Fig. 3, if RNIC0 fails, GPU0-1 will have to traverse across: (1) multiple PCIe SWs, (2) different PCIe root complex/ports, to access other RNICs, resulting in a raw performance drop of up to 64 % due to intra-host performance bottlenecks, which cannot be easily resolved without explicit control at the higher-layer applications.

3 ReCCL

We propose ReCCL, a CCL incorporated with network-fault-tolerant capabilities. ReCCL addresses the shortcomings of the aforementioned approaches in the following way. First, ReCCL is topology and hardware-agnostic. Second, ReCCL does not require external software agents, as the functions are self-contained within the CCL without any external dependencies. Finally, given the CCL’s inherent capability to decide the GPU-GPU communication paths, performance

¹We also engaged with several major vendors in seeking solutions (e.g., specialized RNIC drivers) for failover capabilities like LubeRDMA. However, based on the feedback, concerns surround whether such a solution can remain compliant with the Infiniband standards, especially when message ordering properties, such as atomicity, can no longer be guaranteed when connection states span across multiple RNICs. Even if feasible, such a solution will certainly require additional external software agents that span across the cluster, similar to that of LubeRDMA, which is undesirable.

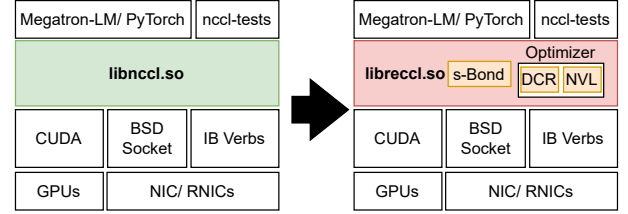


Figure 4. ReCCL is designed to be incorporated within existing CCLs while maintaining compatibility, making it a drop-in replacement. Here, we show the case for NCCL.

optimizations can be performed to overcome intra-host performance bottlenecks.

ReCCL comprises the following. First, to support failover, the ReCCL performs software bonding (s-Bond) to abstract the presence of multiple RNICs for each GPU. Second, to minimize performance degradation during failover, we introduce two optimizations: (1) uses dynamic channel rebalancing (DCR) to reallocate communication channels through the channel scheduler to balance the workload while minimizing bottlenecks, and (2) uses a transit GPU over high-speed interconnects (referred to as NVL) to exploit high-speed GPU interconnects (e.g., NVLink) to reroute traffic through intermediate GPUs, alleviating PCIe bottlenecks.

During initialization, the CCL sets up the primary-backup paths for each inter-host GPU-to-GPU path. Next, depending on the communication group, logical topology, and system characteristics, ReCCL’s performance optimizer dynamically selects the most suitable optimization approach.

Fig. 4 illustrates where ReCCL sits in the distributed AI training software stack. ReCCL is designed to be incorporated within existing CCLs (e.g., NCCL) and is dynamically loaded as a shared library. Thus, existing training frameworks do not need to be recompiled to use ReCCL. ReCCL is designed to interact with the job scheduler to enforce the condition for failover at eq. (4). However, the design of the job scheduler is beyond the scope of this paper.

We detail ReCCL’s failover and recovery scheme introduced in Soft Bonding (§3.1). Then, we discuss performance optimization techniques, including DCR (§3.2.1) and NVL (§3.2.2). Finally, we present implementation details in §3.3.

3.1 ReCCL: Software Bonding

For the CCL to be resilient against network faults, we need network-level redundancy. Existing approaches, such as hardware bonding [26], provide redundancy through the use of dual-port RNICs, where both ports are aggregated as one, and require driver support [35]. However, such approaches require dual-port RNICs – most deployments [13, 33, 36, 51] (including ours) use single-port RNICs, and hardware bonding is still susceptible to RNIC failures.

Thus, we propose a software-based approach – software bonding (s-Bond). s-Bond introduces a failover mechanism

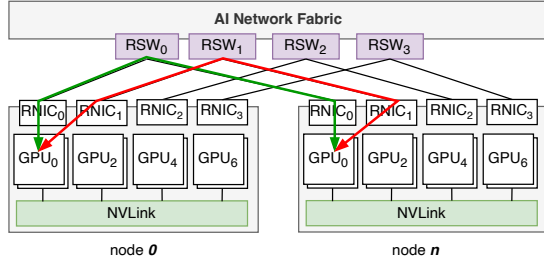


Figure 5. Simplified example of how GPUs are associated with their primary and backup RNICs. The red line indicates the backup path used for failover in ReCCL. The RNICs are interconnected via a rail-based network topology [27, 41, 51], which is common for AI training clusters.

within the CCL, allowing inter-host GPU-to-GPU communications to seamlessly switch over from one RNIC to another when network faults occur. This is feasible given the fact that modern GPU systems come equipped with multiple RNICs. Additionally, such a software-based approach decouples ReCCL from any hardware and driver dependency.

s-Bond operates as follows: Each inter-host GPU-to-GPU pair has a primary and backup path through two RNICs, designated as the primary and backup RNICs. During normal operations, only the primary path is used for GPU-to-GPU communications. Upon detecting a failure in the primary path, the CCL performs a failover to switch to the backup path to retry any previously failed transmissions and resumes subsequent operations. If the primary path recovers² (e.g., after physical link replacement), the CCL triggers the recovery procedure to switch back to its primary RNIC.

Challenges. Realizing s-Bond is non-trivial due to two key challenges. First, the backup RNIC and communication path must be carefully chosen to minimize performance degradation. Poor selection can lead to resource contention within and between hosts, impacting GPU-to-GPU communication efficiency. Second, maintaining communication state consistency is complex, as the sender and receiver may perceive network events differently during failover and recovery.

3.1.1 Provisioning Backup Resources. Provisioning the backup path requires careful consideration of the physical inter-host network topology and the intra-host topology to minimize performance bottlenecks. Key considerations include: (1) how to select the backup RNIC, (2) how to choose the backup communication path, (3) when to set up the backup path. We address this through the following.

Backup RNIC selection. To select a backup RNIC³, we quantify the relative distance between each GPU and available RNICs using the following metric, which is based on their relative performance and physical distance:

- +1 for each PCIe switch traversal,
- +100 for each PCIe root complex (RC) traversal, and
- +200 for each NUMA node traversal.

Using this metric, we identify the RNIC with the shortest distance as the backup. On systems where multiple RNICs share a PCIe switch, we impose a constraint on the number of GPUs that can be assigned to each RNIC to serve as a tie-breaking mechanism to ensure load balance.

Failover communication path. Rather than establishing a mesh between the sender and receiver for each primary and backup RNIC, we adopt a backup-to-backup association: the sender’s backup RNIC is paired with the receiver’s backup RNIC. Thus, when either of their primary paths is unavailable, both the sender and receiver switch to using their respective backup RNICs simultaneously.

This approach aligns with rail-based/rail-optimized network topologies typically used in AI training clusters [27, 41, 51]. Rail-based/rail-optimized network topologies connect GPUs of the same index on different nodes across racks via per-index rail switches, enabling single-hop, low-latency GPU-to-GPU communications. By aligning with the network topology’s property, we keep failover communication confined within the same rail – avoiding cross-rail traffic, which could introduce additional latency due to extra hops. Additionally, the homogeneous nature of GPU systems ensures that both the sender and receiver automatically align on their failover communication paths. This simplifies failover handling and guarantees consistent communication paths.

Backup path setup. We proactively set up the backup path during the CCL initialization phase, instead of reactively when network faults occur. This decision enables us to piggyback on the existing communication setup process, eliminating the need for additional communication rounds to configure the backup RNICs. Therefore, the backup path remains on “standby”, ready to handle failover events immediately when required.

Having the backup path ready has an additional benefit: we can make use of it for in-band signaling purposes to synchronize the sender and receiver during failover. We elaborate more on this in the subsection that follows (§3.1.2).

3.1.2 Keeping Communication States In-sync. Maintaining consistent communication states between the sender and receiver during failover and recovery is critical. Both

²In the case where the RNIC fails, recovery is not possible.

³In practice, ReCCL can provision multiple backup RNICs or paths, depending on user configuration. This ensures ReCCL can tolerate multiple switch-host link or RNIC failures. For simplicity, we assume that only one backup RNIC is provisioned in the remainder of the paper.

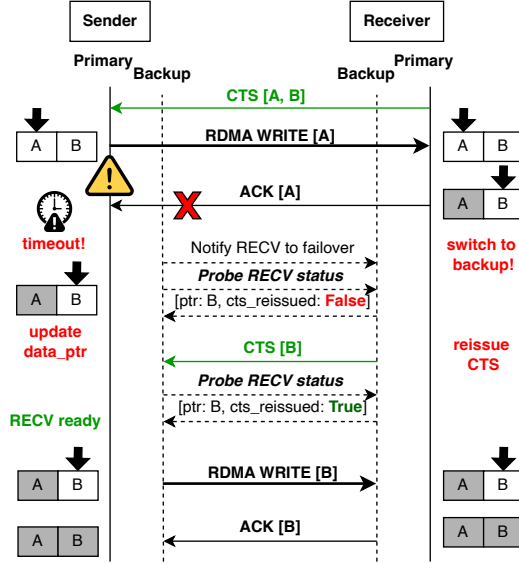


Figure 6. When the sender detects a network fault on the primary path, it notifies the receiver and initiates failover. It then checks if the receiver has switched to the backup path and reissues the CTS for pending requests. Once the receiver is ready and both data pointers are aligned, communication resumes over the backup path.

maintain data pointers that must advance in sync to ensure correct communication and computation. This requires meeting two conditions:

1. both sender and receiver must use the same path, i.e., operate on the same RNIC (primary or backup), and
2. both sender and receiver must advance their data pointers in sync to avoid inconsistency.

However, failover and recovery events can introduce ambiguities. Below, we provide examples of how this can manifest in practice:

- **ACK lost:** During a network fault, the receiver may successfully process a message, but the sender may not receive an ACK. The sender retransmits via the backup path, but the receiver, unaware of the failover, has no RDMA Receive Requests (RRs) posted on its backup RNIC. This results in an “RNR Retry Counter Exceeded” error at the sender.
- **Data pointer mismatch:** Even if the receiver has switched over to the backup path and has RDMA RRs posted, if ACK losses remain unresolved, the sender may retransmit stale data (say, data n), while the receiver has advanced to the next data (say, data $n + 1$). The stale data erroneously consumes the RDMA RR⁴ intended for data $n + 1$, leading to the receiver falsely assume that data $n + 1$ has been received. This misalignment causes incorrect computation results and eventual task failure.

To that end, ReCCL proposes the following procedures:

⁴In RDMA, RRs are simply consumed by any incoming RDMA Write with Immediate Data requests. The corresponding memory addresses that are written to are not verified [6].

Failover procedure. To keep the sender and receiver in sync during failover, we introduce an in-band synchronization phase before failover. We illustrate this in Fig. 6 using one of the failure scenarios, i.e., when a sender’s primary RNIC goes down (e.g., due to link failure).

Sender. Upon detecting a failure at the primary path, the sender first performs the following two steps before switching over to the backup path for subsequent (re-)transmissions:

1. **Notify the receiver:** The sender signals the receiver of an impending failover via the backup path.
2. **Probe receiver state:** The sender continuously probes the receiver’s data pointer to check if the receiver has switched to the backup RNIC. Only when the receiver reaches a stable state (all pending requests reissue their CTS on the backup RNIC) does the sender stop probing and resume retransmission on the backup RNIC. During retransmission, the sender verifies data pointer alignment to avoid duplicate transmissions.

Receiver. The receiver triggers the failover procedure when:

1. **Notification from sender:** Upon receiving a failover notification, the receiver reissues a CTS for all pending requests on the backup RNIC.
2. **Failed to send CTS:** If unable to send CTS via the primary RNIC, the receiver reissues a CTS for all pending requests on the backup RNIC and notifies the sender (through a flag in the CTS message) to switch over to use the backup RNIC. The receiver transitions to a “ready” state once it reissues the CTSes. This is signaled by an internal variable for which the sender can probe.

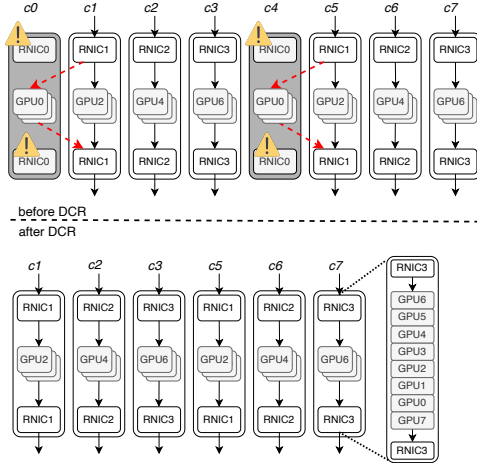
Recovery procedure. Recovery involves transitioning back to the primary path when it returns to being operational.

Sender. Upon detecting recovery of the primary RNIC, the sender notifies the receiver and waits for the receiver to signal readiness through a recovery flag in subsequent CTS messages. Once flagged, the sender switches back to using the primary RNIC for all future transmissions.

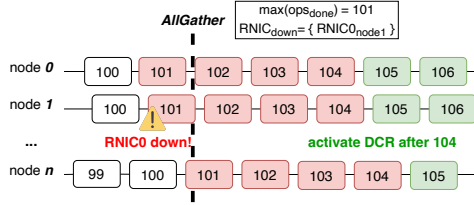
Receiver. The receiver detects the primary path’s recovery through a notification from the sender or by monitoring its local primary RNIC status. It sets a recovery flag in the next CTS message sent via the backup RNIC, signaling the sender to switch back after the current CTS. All subsequent CTS messages are then posted on the primary RNIC.

3.2 ReCCL: Performance Optimizations

After failover, a drop in network communication performance is expected given the shared bandwidth resources on the backup RNIC, as well as the presence of intra-host bottlenecks on certain system configurations. We present two approaches to improve communication performance after failover, one for small and one for large communication groups. During CCL initialization, the performance optimizer decides on either one of the corresponding optimizations to use, whenever feasible.



(a) Before DCR, c0 and c4 share RNIC1 with c1 and c5 during failover. The sharing of RNIC1 causes c0, c1, c4, and c5 to be the straggler channels among all. After DCR, c0 and c4 are suspended, restoring c1 and c5's performance.



(b) Periodic *AllGather* operations are used to distribute system statuses and their total completed collective operations (ops_{done}) among each node to determine DCR activation.

Figure 7. DCR illustration.

3.2.1 Dynamic Channel Rebalancing. After a failover, a drop in communication performance can result in straggler channels, i.e., specific CCL channels experiencing larger delays, causing uneven performance across parallel channels. To address this, we propose Dynamic Channel Rebalancing (DCR), a technique to mitigate performance degradation by selectively disabling impaired channels (i.e., those relying on backup RNICs).

CCLs typically use multiple channels in parallel by splitting data across channels during operations to saturate the network to maximize communication performance. When a channel is impaired, DCR dynamically excludes it from use, redistributing data across the remaining healthy channels to minimize straggler-induced delays by balancing communication workloads among the channels. Once the impaired channel recovers, DCR reintegrates it, restoring the full set of channels for subsequent operations.

Take Fig. 7a as an example, we see that when one RNIC fails (in this case, RNIC0), 2 channels become impaired (c0 and c4), leaving 6 healthy channels. With DCR, the 2 impaired channels are disabled, and data are redistributed equally among the 6 healthy channels by the channel scheduler.

Dynamic Channel Rebalancing. DCR must be applied consistently across worker nodes – each worker node has to use the same set of CCL channels for communication at any point in time, otherwise, it results in the CCL to hang. To keep all worker nodes in sync for DCR, we periodically perform an *AllGather* operation to distribute the system states (e.g., RNIC status) and also the total number of completed collective operations (ops_{done}) among each node. Based on the gathered system states, each node checks for impaired channels. A channel is impaired if it consists of an RNIC that is currently in downstate in any node. Impaired channels are excluded from later collective operations.

To ensure consistency, all nodes should apply DCR simultaneously, delaying impaired channel exclusion until a synchronization point. Thus, DCR is only performed after a pre-defined offset of number of collective operations. This ensures a sufficient gap for all nodes to complete any ongoing collective operations before activating DCR. From the example shown in Fig. 7b, the current largest ops_{done} is 101. With an offset of 3, DCR will take effect after $ops_{done} = 104$, i.e., from 105 and beyond. A similar process is applied when the primary RNIC of an impaired channel recovers.

Why not use DCR to handle failures directly? We apply DCR only after s-Bond completes failover, but one might ask: could DCR handle failures directly, without relying on s-Bond? While conceptually possible, directly applying DCR to failures would require substantial modifications to the channel scheduler. Specifically, the scheduler would need to precisely track which data has been successfully sent and which remains unsent to manage retransmissions accurately at the channel level. Furthermore, the retransmission logic would then need to be globally synchronized across all GPUs participating in collective communication operations, significantly complicating the implementation and potentially introducing performance overhead. Due to the added complexity and synchronization costs involved, we opt instead for the simpler and more practical approach: first allowing s-Bond to handle failover and subsequently using DCR to optimize performance by disabling impaired channels.

Limitation(s). DCR is only applicable when a CCL communication group consists of multiple channels using disjoint RNICs, as there are no noticeable benefits of performing DCR when all channels share a common RNIC (e.g., in small communication groups). For cases when DCR is not feasible, we present our approach that makes use of the high-bandwidth GPU interconnect to maximize communication performance in the section that follows (§3.2.2).

Additionally, as DCR disables impaired channels to avoid stragglers, when there are multiple network failures, more channels are disabled, thus leaving only a handful of healthy channels. In such a case, the remaining healthy channels will saturate, and performance gain shrinks. As pointed out in eq. (3), failover is only preferred when the slowdown

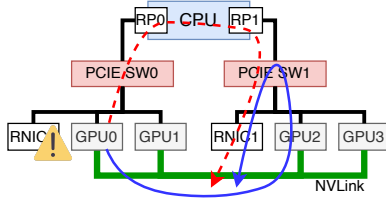


Figure 8. GPU-to-backup RNIC communication using a transit GPU: Data from GPU0 is routed through GPU2 over high-speed GPU interconnects (green paths) to mitigate the PCIe bottleneck (red dashed paths) when RNIC0 is unavailable.

is lower than the restart overhead. In extreme cases with multiple network failures, a restart may be the better option.

3.2.2 Using a Transit GPU over High-speed GPU Interconnects. When DCR is not feasible, the high-speed GPU interconnect (e.g., NVLink) provides another option to overcome the intra-host PCIe bottlenecks (we refer to this as NVL thereafter). The use of high-speed GPU interconnects and intermediate GPUs to optimize AlltoAll communication performance in rail-based topologies is well-documented [30]. In ReCCL, we adapt this technique to address performance challenges during network faults.

Refer to Fig. 8 as an example setup. During the CCL initialization phase, additional memory is provisioned on the transit GPU (i.e., GPU2) as a transit buffer (tens of MB), which is associated to the RDMA QPs on the backup RNIC (i.e., RNIC1). To send data from GPU0 via RNIC1: First, GPU0 performs a device-to-device memory copy to GPU2 over the high-speed GPU interconnect. Then, once the copy is complete, an RDMA WRITE is posted to RNIC1, sending data from GPU2’s transit buffer. At the receiving node, this process occurs in reverse, with data arriving at the transit GPU before being copied to the destination GPU.

Trade-offs and resource overhead. The overhead is the memory required for transit buffers, with each consuming several MBs. As we use only GPU memory (not SMs), NVL does not interfere with training compute. To minimize impact, we evenly distribute required transit buffers across GPUs located within the same PCIe switch. As this method is typically employed in small communication groups when DCR is not possible, the additional resource required for NVL is minimal in practice.

3.3 ReCCL: Implementation

We have integrated ReCCL with NCCL v2.19, a widely used NCCL version in our current deployments. At a high level, applying ReCCL to NCCL requires modifications primarily to two key source files, namely: (1) `net_ib.cc`: changing the data and control paths to implement s-Bond⁵ and

⁵Instead of modifying `net_ib.cc`, we also implemented a standalone NCCL NET plugin, which is a dynamically loaded shared library that incorporates s-Bond only. Using the standalone plugin, this allows us to provide s-Bond support to other NCCL versions for trials.

NVL, and (2) `enqueue.cc`: modifying `addCollToPlan` to implement the DCR logic. Minor changes were also made to `init.cc` and `bootstrap.cc` to support the periodic All-Gather (Fig. 7b) used to distribute system statuses for DCR.

To use ReCCL, neither code changes nor recompilations are required for the training framework. Existing training frameworks can simply use ReCCL as a direct replacement (Fig. 4) for the corresponding NCCL versions. At the time of writing, we have also ported ReCCL for NCCL v2.21 and are actively extending support for newer NCCL versions. Additionally, there are also ongoing efforts to incorporate ReCCL’s failover mechanism and optimizations beyond NCCL to support non-NVIDIA GPU clusters.

In the following, we highlight additional implementation details not covered in our earlier design.

s-Bond. Whenever a failover is triggered, we make sure that all the QPs associated with the primary RNIC are transitioned to the ERR state. This is to ensure that there are no WRs left in the queue to ensure a clean transition to the backup RNIC. Similarly, during recovery, before switching back to the primary RNIC, we ensure the primary RNIC on both sender and receiver is reset and returned to the RTS state. At the sender, to probe the receiver’s status, we perform RDMA READs to continuously monitor the receiver’s readiness to initiate failover and subsequent retransmissions.

Fast-failover. In CCLs, different communication groups and their respective channels often share the same RNICs, but not all groups are active when a network fault occurs. Idle channels may remain unaware of RNIC failures until they attempt communication and experience a timeout, triggering failover separately. To mitigate this delay, we maintain a global RNIC state that is updated whenever an active channel detects a failure. This allows previously idle channels to immediately switch to the backup path without waiting for a timeout, enhancing system performance and reducing failover latency. The same applies to recovery.

DCR. We introduce a separate thread to periodically execute the all-gather operation to gather system states on each node. This operation is done over the out-of-band network, specifically, we repurpose the `bootstrapAllGather` primitive used during CCL initialization. For DCR to dynamically redistribute data across the channels, we perform minor modifications to the channel scheduler function – `addCollToPlan`, to enable it to react accordingly to the gathered system states.

NVL. We provision the transit GPU buffers when the primary GPU registers its memory buffer with its primary RNIC. During failover, the sender and receiver keep track of two sets of pointers, one for the primary GPU and the other for the transit GPU, to ensure data is delivered to the correct offset. As we use NVIDIA GPUs, memory copies across different GPUs via NVLink are asynchronous – we have to perform

explicit device synchronization to ensure its completion before proceeding to post the RDMA WR on the sender, or flag the request completion on the receiver.

To prevent potential out-of-memory issues, we implement a user-configurable threshold to cap the total amount of GPU memory that can be allocated as NVL transit buffers. If the threshold is exceeded, NVL is thus not applied for the subsequent channels. We apply this best-effort approach in allocating the transit buffers (similar to that of FuseLink [44]) because estimating GPU memory usage remains a hard problem, as highlighted by Gao et al. [14].

4 Evaluation

First, we study the gains of performing failover using ReCCL on end-to-end training time through simulations in §4.1 to highlight the performance gain of having ReCCL. Next, using a testbed with 32 GPUs, we evaluate the training performance using ReCCL during failover in §4.2.1 and the effectiveness of ReCCL's individual optimizations in §4.2.2.

4.1 Software Simulations

Due to resource availability at our production cluster, we perform software simulations to study how failover using ReCCL can improve training efficiency in an end-to-end training and large-scale setting in the presence of network faults at the last hop between a switch and a host. In particular, we compute the total number of GPU hours required, which can be directly correlated with the training costs.

In our simulations, we assume a typical training scenario where the training goes epoch by epoch, and checkpointing is performed regularly at $T_{interval}$. We simulate failures on the RNIC-ToR links and ToR switches during training and compute the expected total training time required. We use the Llama3-405B training [28] as our case study target; this training workload uses 16k GPUs and requires a training time of 1925 hours [28]. The default parameters used in the simulations are summarized in Table 1.

Throughout the simulations, we follow the formulations defined in §2.1.1 for restart and failover cases (i.e., training runs at a degraded state until the next checkpoint before restarting), unless otherwise specified. We also take into consideration that there can be more than 1 network fault occurrence during the failover state. Each simulation is repeated for 10,000 iterations.

We compare ReCCL against the following baselines:

- **fault-free:** training completes with no error. This is the best-case scenario with the minimum training time.
- **restart:** training restarts immediately from the last checkpoint with the same number of nodes.
- **restart+scaling:** training restarts immediately from the last checkpoint with the faulty DP group removed. The faulty nodes are replaced after 24 hours. We set the DP dimension to be 16 as per [28].

Table 1. Default simulation parameters.

Parameter	Definition	Default Value(s)
$T_{interval}$	Checkpoint interval	1.0 hr [16]
$T_{timeout}$	RNIC ReTX timeout	1 minute
T_{detect}	Training framework/ watchdog timeout	0.5 hr [16]
T_{reload}	Overhead to restart a training job	0.5 hr [16]
k	Training slowdown rate during failover	1.03 (1 failure) 1.20 (2 failures) 1.50 (3 or more)
λ_{link}	Monthly RNIC-ToR link failure rate	0.057% [41, Sec. 2.3]
λ_{switch}	Monthly ToR switch failure rate	0.051% [41, Sec. 2.3]

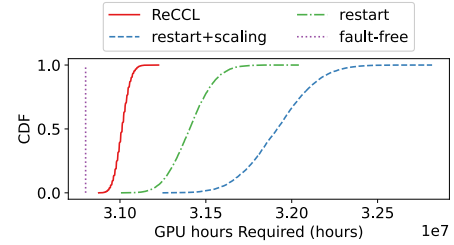


Figure 9. Impact of different fault-handling strategies: ReCCL (failover), restart, and restart+scaling compared to the fault-free scenario, on the total GPU hours required for Llama3-405B training with 16k GPUs.

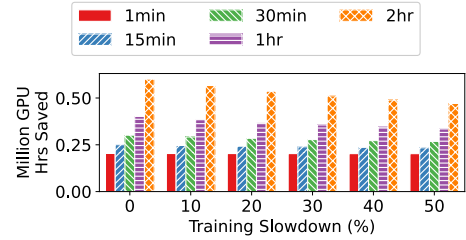


Figure 10. Saved GPU hours using ReCCL compared to the restart case under different training slowdown ratios and checkpoint intervals.

Impact on end-to-end training time. First, we compute the end-to-end training time required in the presence of network faults. From the CDF in Fig. 9, we observe that using ReCCL, one can effectively reduce the total number of GPU hours required to complete the training in the presence of network faults. Compared to restart and restart+scaling, ReCCL reduces the median additional GPU hours over fault-free training by 64.96 % and 80.97 %, respectively. When we consider the 99th percentile, the reductions are 65.43 % and 80.08 %, respectively.

These results indicate that ReCCL can potentially improve training efficiency substantially.

Impact of different training slowdown rate and checkpointing interval. In Fig. 10, we show the impact of how the training slowdown ratio (i.e., the k value) can impact the potential gains of using ReCCL. k varied from 1.0 to 1.5. At the same time, we also show the case for different checkpointing intervals ranging from 1 minute to 2 hours. We compare it against the ideal restart case when the number of available training nodes remains the same throughout, without scaling.

Across the board, we can observe that as the training slowdown increases, the potential gain decreases. This is because the operating regions, as per eq. (4), to perform failover using ReCCL have become smaller, which is expected. Nevertheless, even at a 50 % slowdown (while unlikely), the total number of GPU hours that can be saved can still be more than 200K GPU hours even when using an aggressive 1-minute check-pointing interval. This substantiates the benefits of having the option to perform a failover instead of simply restarting.

4.2 Testbed Evaluations

Next, we evaluate ReCCL using a small-scale testbed to understand its performance during failover. It is important to note that, as a CCL, ReCCL is agnostic to both the underlying network topology and the AI models trained.

Testbed setup. Our testbed consists of 4 GPU nodes with 32 GPUs in total. Each node comprises 8 NVIDIA Hopper GPUs (96GB HBM3) coupled with 4 NVIDIA BlueField-3 SuperNIC 400G RNICs; all GPUs of the same node are also interconnected with NVLink (similar to the system in Fig. 3). The GPU nodes are interconnected via a rail-optimized topology (similar to Fig. 5, the RNICs on each GPU node are connected to four different rail switches) using a RoCEv2 network fabric with 51.2Tbps Tomahawk5 [8] ASIC-based switches.

4.2.1 Training performance. We investigate how failover impacts model training performance. We use the Megatron-LM [33] training framework and measure the average slowdown per training iteration after failover. This slowdown is related to the k value in our formulations presented earlier (§2). As $k \geq 1$, the slowdown is measured as the (normalized) additional time needed and is ≥ 0 . We use two variants of ReCCL, one consisting of s-Bond only, and the other comprising all optimizations discussed in §3.2.

Given that different models have different computation-to-communication ratios (e.g., given the model sizes and parallelism strategies), failover can impact training performance differently. We study the performance when a single link/ RNIC failure is down using BEVFormer-V2 [54], and Llama2 [50] with three different model sizes (7B, 13B, and 70B)⁶. For Llama2, unless otherwise specified, we use the recommended parallelism strategies for our 32-GPU testbed setup: Llama2-7B (TP=1, PP=1), Llama2-13B (TP=8, PP=1), and Llama2-70B (TP=8, PP=2).

Fault-free training performance. First, we benchmark the training performance of ReCCL against the vanilla NCCL in the absence of failures. Across multiple training runs across the models mentioned above, the training performance (in terms of time taken per iteration) of ReCCL when compared to NCCL at the median, and 99th percentile remains within ± 0.5 % of each other. This indicates that their

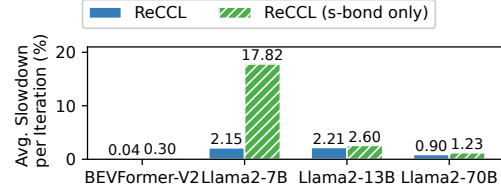


Figure 11. Average slowdown in the time taken per training iteration when network faults occur. Note: lower is better.

performance is identical and ReCCL does not incur additional overhead to training. Henceforth, we will only report the results for failover performance.

Training performance during failover. Next, we measure the training performance during failover. We depict the results in Fig. 11. We see that ReCCL’s failover approach ensures that the training continues seamlessly (without restart) after a network failure. Furthermore, we also observe that ReCCL’s optimizations noticeably improve the training performance during failover. With respect to the model presented in §2.1.1, k is between 1.004 and 1.0215.

An interesting case to note is for Llama2-7B, where the performance difference between using the two ReCCL variants can be up to 15 %. This is due to: (1) for a smaller model like Llama2-7B, its compute-to-communication ratio is smaller as compared to its 13B and 70B counterparts, and (2) the parallelism strategy used for Llama2-7B in this experiment is data parallel-only – more data is transferred among GPUs.

On the other hand, while the relative increase in training time per iteration for Llama2-70B appears smaller compared to the other Llama2 variants, the absolute increase was much larger. For instance, it can be up to 4.36 $\times$ when compared to the Llama2-7B. This can be explained by Llama2-70B’s baseline training time per iteration being an order of magnitude higher than that of Llama2-7B, resulting in a smaller relative difference despite the larger absolute change.

Impact of parallelism strategies. Next, we look into how different training parallelism [33, 53] strategies like tensor parallel (TP) and pipeline parallel (PP) can impact the training performance during failover. Given that Llama2-7B can be fully data parallel (i.e., TP=1, and PP=1) due to its smaller model size, it allows us to fully explore how TP or PP impacts the performance. Therefore, we only show the results for Llama2-7B.

We perform two evaluations by keeping either TP or PP to 1 and varying the other parameters from 1 to 8. The results are shown in Fig. 12. In both cases, slowdown decreases with a larger amount of parallelism (TP or PP). When TP=1, training performance stabilizes near zero for $PP \geq 4$. Similarly, with PP=1, increasing TP also reduces slowdown. These trends align with expectations, as reduced GPU data exchange and minimal PCIe bottlenecks limit performance impact. Across the board, ReCCL with its optimizations keeps the training slowdown low, demonstrating its effectiveness.

⁶ReCCL has also been tested with other open (e.g., Qwen [7]) and internal proprietary models showing similar training performance degradations during failover, but they are omitted in this paper for brevity.

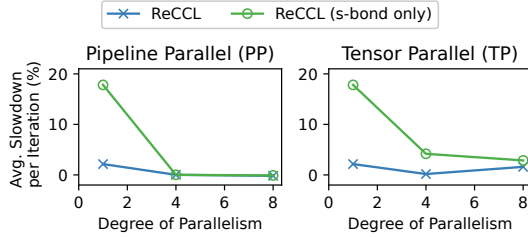


Figure 12. Impact of different parallelism strategies on the failover performance for Llama2-7B. Note: lower is better.

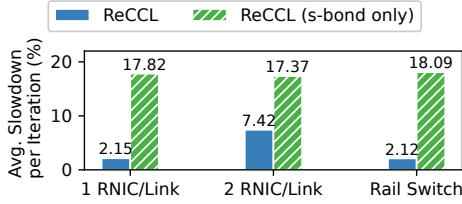


Figure 13. Impact of different types of network faults. Here, we show the case for Llama2-7B. The trend generalizes to other models. Note: lower is better.

Impact of different network faults. Following, we study how different types of network faults can impact failover performance. We only show the results for Llama2-7b in Fig. 13, given its more pronounced slowdown shown earlier in Fig. 11; the trend generalizes for other models. We evaluate the following three scenarios: (1) one RNIC/link down, (2) two disjoint RNIC/link down, and (3) rail switch outage.

First, the results for 1 RNIC and rail switch failure are very close because the CCL channels impacted are the same: a rail switch outage takes down RNICs of the same index across all GPU nodes – the same set of CCL channels are affected. Next, when using s-Bond only, we observe similar failover performance across all cases, with a slowdown of approximately 18 %. The similar slowdown in performance is due to the communication performance being impacted by straggler channels, which happen to have the same performance. In the case of ReCCL, we observe the performance slowdown increased by $\sim 5\%$ when two disjoint RNIC/links are down. This is due to the presence of more impaired channels, which are subsequently disabled by DCR, causing the number of parallel channels to be halved as compared to the other two cases, where three-quarters of the channels remain.

Key takeaways. The results show that ReCCL can recover completely from network faults through failover on the testbed. With the presented optimizations, the training slowdown can be kept low, with $k < 1.1$ in most cases. Early trials of ReCCL on a production GPU cluster using 1024 GPUs have also demonstrated its robustness and ability to recover from multiple network faults.

4.2.2 Ablation study. To highlight the impact of the optimizations integrated in ReCCL, we perform an ablation study to see how DCR (§3.2.1) and NVL (§3.2.2) improve collective

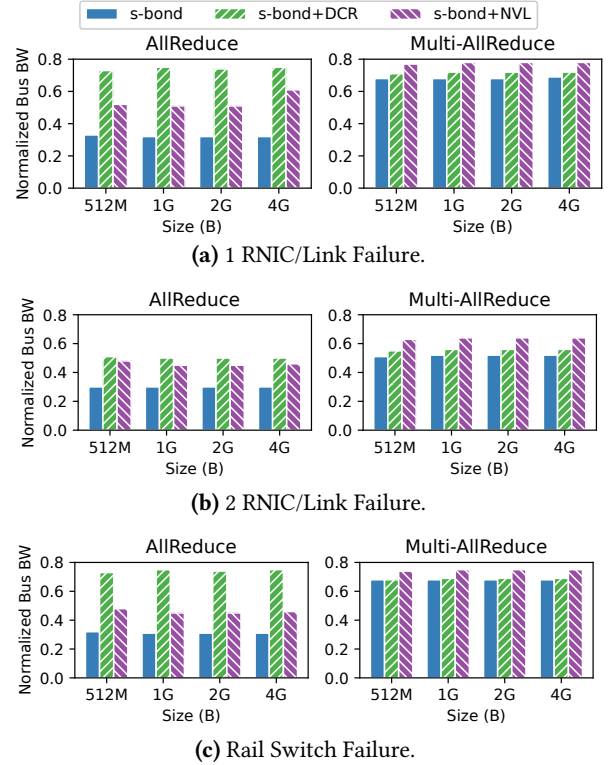


Figure 14. Collective communication performance under network faults is normalized to the fault-free performance. Note: higher is better.

communication performance on top of s-Bond. We use the NCCL tests [38] framework to benchmark the collective communication performance. At the same time, we also compare their GPU memory overhead when running the tests.

Raw collective communication performance. First, we show the normalized bus bandwidth compared to the baseline performance (no network fault). We run two settings: (1) a large communication group, e.g., AllReduce, which involves all RNICs and GPUs with multiple disjoint channels in parallel, and (2) a small communication group, e.g., Multi-AllReduce, where each communication group only has one RNIC (i.e., no disjoint channels), e.g., when PP or TP uses the inter-host network fabric [41, Sec. 9.2].

From the results depicted in Fig. 14, we observe that DCR offers the best performance for AllReduce, followed by NVL and the vanilla s-Bond. As there are multiple disjoint channels, DCR reduces the degree of parallelism to improve performance back to approximately 75 % and 55 % for each 1 RNIC/ Rail Switch and 2 RNIC failure cases, respectively. When using NVL, as we bypass the intra-host PCIe bottlenecks, we achieve about 50 % of the original performance, which is expected, due to RNIC sharing.

As for Multi-AllReduce, NVL performed the best, with a performance improvement margin of up to 11 % as compared to the vanilla s-Bond and DCR. As there are no disjoint channels, DCR is not applicable, and thus, there are no noticeable

performance improvements observed. Through DCR and NVL, our optimizations cover a wide range of collective communication patterns, allowing ReCCL to be performant as much as possible during failovers.

GPU memory overhead. Finally, we contrast the GPU memory overhead of each optimization against the vanilla NCCL. We observe that there is zero GPU memory overhead for both s-Bond and DCR. However, as for NVL, since it requires GPU memory buffers for transit, it incurs an overhead of 30 to 40 MB per CCL channel. Using the earlier AllReduce workload as an example, which by default uses 8 CCL channels on our GPU system, the observed overhead ranges from 281 to 320 MB per GPU, which makes up for less than 1 % of the available GPU memory.

5 When ReCCL matters

Analytical modeling. Beyond training resiliency, it is important to understand how ReCCL impacts system performance. Therefore, we develop a simple analytical model to allow the estimation of the performance gains and evaluate the impact of introducing ReCCL in different setups.

Recall that in §2.1.1, we considered the tradeoff between failover and restart when a network fault has occurred within a checkpoint interval. At the system level, the likelihood of link failure occurring depends on the link failure rate within a checkpoint interval and the number of links between a switch and a host involved in a training job⁷. Given a single checkpoint interval, if each of the n links fails independently with probability q , then:

$$X \sim \text{Binomial}(n, q)$$

Let P be the probability that there is at least one link failure within the interval, then:

$$P = P(X \geq 1) = 1 - (1 - q)^n \quad (6)$$

Next, we denote T_{restart} as the time taken to reach the next checkpoint iteration when there is a link failure and restart is the only option.

$$T_{\text{restart}} = t_{\text{elapsed}} + t_{\text{restart}} \quad (7)$$

Let T_{ReCCL} be the time taken when either failover or restart is used, decided based on the eq. (4). Hence,

$$T_{\text{ReCCL}} = t_{\text{elapsed}} + \min(t_{\text{failover}}, t_{\text{restart}}) \quad (8)$$

When there are no link failures, the time taken to reach the next checkpoint is equivalent to the checkpoint interval itself, and we denote it as T_{healthy} . Therefore,

$$T_{\text{healthy}} = T_{\text{interval}}$$

Assume that failures are uniformly distributed between 0 and T_{interval} . Therefore, $E(t_{\text{elapsed}}) = \frac{T_{\text{interval}}}{2}$. To simplify

⁷To ease modeling, we omit switch failures.

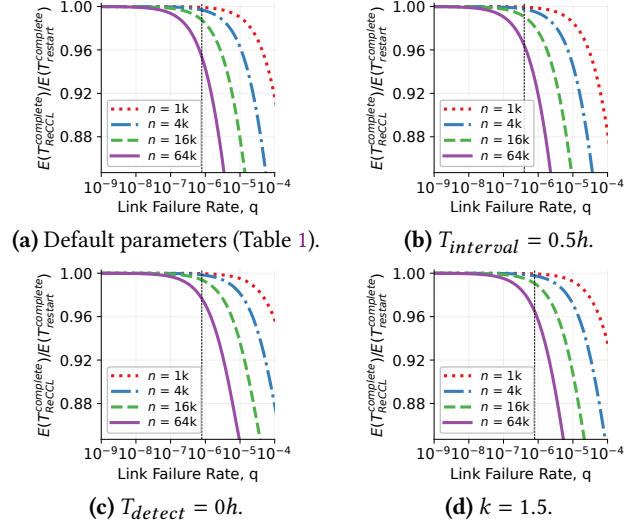


Figure 15. Analytical results computed for varying job sizes (n) and link failure rate (q). The dashed vertical line corresponds to the monthly link failure rate of 0.057% [41] (Table 1). Note: a lower ratio is better.

the model, we assume that the current iteration will always complete successfully after a restart/failover, a reasonable approximation for small P . Finally, the expected completion times for restart and ReCCL can be expressed as follows:

$$E(T_{\text{restart}}^{\text{complete}}) = P(T_{\text{restart}}) + (1 - P)(T_{\text{healthy}}) \quad (9)$$

$$E(T_{\text{ReCCL}}^{\text{complete}}) = P(T_{\text{ReCCL}}) + (1 - P)(T_{\text{healthy}}) \quad (10)$$

Note that $E(T_{\text{ReCCL}}^{\text{complete}}) \leq E(T_{\text{restart}}^{\text{complete}})$ since ReCCL is adaptive and always chooses the more effective option.

Estimating potential gains. To study the potential gains across different setups, using the earlier presented model, we vary the link failure rate (q) and job sizes (n) to observe their impact on $\frac{E(T_{\text{ReCCL}}^{\text{complete}})}{E(T_{\text{restart}}^{\text{complete}})}$. Unless otherwise stated, the parameters used are consistent with those listed in Table 1. We highlight some of the computed results in Fig. 15.

Overall, the benefits of ReCCL scale with both the size of the training job and the underlying link failure rate. At a given monthly link failure rate of 0.057% [41] ($q = 7.92 \times 10^{-7}$), we observe the potential performance gains across different job sizes. Using Fig. 15a as the reference, we observe that the gains are 0.08%, 0.32%, 1.25% and 4.63% for 1k, 4k, 16k and 64k-sized jobs, respectively. From the results, the expected gains for smaller job sizes, in this case, up to 4k GPUs, are less than 1%. In such cases, the additional complexity of ReCCL may outweigh the benefits, but for larger job sizes, ReCCL can yield noticeable gains. This is highly relevant given the trend of increasingly large training jobs [55], especially for the development of state-of-the-art frontier models, which are known to be time-consuming and

costly. To put things in perspective, at a modest GPU rental cost of ~\$800 per month, training on 64k GPUs for three months costs ~\$150M. A ~4% reduction, as suggested by our analysis, yields savings of several million dollars. This highlights the potential gain of deploying ReCCL.

For smaller-sized jobs, e.g., 1k and 4k, Fig. 15a shows that the expected gains for using ReCCL start to exceed 1% only when the monthly link failure rate goes beyond 0.73% ($q = 1.02 \times 10^{-5}$) and 0.18% ($q = 2.54 \times 10^{-6}$). While these failure rates are higher than the average failure rates reported in previous studies [41], it has also been observed that link/switch failures can exhibit high variance [47] and that a small number of weak links can fail at a disproportionately higher rate than all other links [15]. As such, under such circumstances, ReCCL can offer noticeable benefits.

Using the larger-sized jobs, i.e., 16k and 64k, as examples: when a smaller checkpoint interval is used ($T_{interval}$ reduces from 1hr to 0.5hr), the relative improvement sees a slight reduction from 1.25% and 4.63% to 0.94% and 3.57% (see Fig. 15b). From Fig. 15d, we also observe a similar slight reduction when the training slowdown factor, k , increases from 1.03 to 1.5. On the other hand, the impact on ReCCL's relative improvement is more significant when the training watchdog timer, T_{detect} , is reduced from 0.5hr to 0, resulting in a reduction from 1.25% and 4.63% to 0.62% and 2.33%, respectively (see Fig. 15c). Nevertheless, for large training jobs ReCCL remains relevant.

6 Discussion

ReCCL production rollout. In [41], the authors claimed that a custom CCL like ReCCL would be difficult for production rollout. From our experience, it is the contrary, for two reasons: (1) the fruition of ReCCL came through our continuous engagements with our internal teams and external partners to minimize the impact of network faults, and (2) ReCCL neither requires changes to existing training frameworks nor code recompilation (see §3). ReCCL is currently undergoing trials in our production clusters, with a phased rollout expected to continue throughout the year.

Planned network maintenance. Beyond network faults, ReCCL makes planned network maintenance, such as in-place switch upgrades or (preventive) “hot” link replacements, less disruptive, given how training jobs can remain running – significantly improving the flexibility for our network operations team. During maintenance, we can explicitly configure the job scheduler to delay/avoid restarting existing jobs if the necessary maintenance can be completed within the stipulated service-level agreements. This will be supported in the future as we gradually roll out ReCCL.

7 Related Work

Distributed AI training fault handling. Restarting training jobs upon *any* failure events and replacing faulty

nodes with healthy worker nodes [16, 20, 57], or reducing the training scale [24, 46], are today's common practices. Upon restart, training resumes from the previous checkpoint [19, 43, 45]. Instead of indiscriminately restarting training jobs upon faults, ReCCL allows network faults to be handled differently, allowing training jobs to seamlessly failover (with the best possible performance) and preserve training progress. The failover mechanism provided by ReCCL is beneficial even with faster checkpointing mechanisms (like CheckFreq [32], Check-N-Run [11], Gemini [52] and [37]), and rapid error detection mechanisms (e.g., [9, 16, 20, 37]).

Apart from restarting, emerging proposals posit that one could allow training to continue with faults even at degraded states. ReCCL aligns well with this trend, and is the first to do so in handling network faults, while existing work like Oobleck [18], Bamboo [48], and ReCycle [12] either focus on worker node failures or GPU failures. ReCCL is orthogonal with them, and can work hand-in-hand towards the goal of realizing a holistic, resilient distributed AI training stack.

RDMA/ network fault tolerance/ multi-path. There is extensive research [17, 21, 34, 42, 56] on network faults occurring between switches; however, these faults rarely cause the CCL to lose connection, leading to subsequent training job failures, due to the rich path diversity in data centers, and thus are out of our scope. For the last hop between a switch and a host, existing solutions like hardware bonding [26, 35, 41] aggregate dual-port RNICs to enhance throughput and fault tolerance, yet they remain vulnerable to RNIC failures. Approaches such as MPRDMA [29], Virtuoso [49], FuseLink [44], and NCCL's recent MERGE_NIC option leverage multiple paths/ RNICs for communication, but do not handle failures at the last hop between a switch and a host; thus, the CCL and training job remain vulnerable to network faults. On the other hand, recent work like LubeRDMA [25] provides userspace IB Verb failover with a primary-backup mechanism; however, as highlighted in §2.3, it introduces deployment complexity and single points-of-failure due to external coordination agents, both of which ReCCL avoids.

8 Conclusion

We present ReCCL, the first network fault-tolerant CCL. With ReCCL, network faults can now be handled differently, instead of being naively restarted, which results in wasted compute. ReCCL enables us to better handle network faults for large-scale distributed AI training jobs, and it is gradually being rolled out in production. ReCCL contributes a key piece to the puzzle towards robust and efficient distributed AI training in the global pursuit of artificial general intelligence.

Acknowledgments

We thank the reviewers and our shepherd, Philip Levis, for their invaluable feedback in improving this paper.

References

- [1] [n. d.]. <https://github.com/NVIDIA/nccl/issues/998> [Accessed: Oct 2024].
- [2] 2024. ACCL: Alibaba Cloud's self-developed high-performance collective communication library. <https://help.aliyun.com/zh/pai/user-guide/accl-alibaba-high-performance-collective-communication-library> [Accessed: Jan 2025].
- [3] n.d. Infiniband Verbs Performance Tests. <https://github.com/linux-rdma/perftest> [Accessed: Jan 2025].
- [4] AMD. n.d. ROCm Communication Collectives Library (RCCL). <https://github.com/ROCm/rccl> [Accessed: Jan 2025].
- [5] AMD. n.d. What is RCCL? <https://rocm.docs.amd.com/projects/rccl/en/latest/what-is-rccl.html> [Accessed: Jan 2025].
- [6] InfiniBand Trade Association. 2020. InfiniBand Architecture Specification Release.
- [7] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023).
- [8] Broadcom Inc. [n. d.]. Tomahawk 5 / BCM78900 Series - 51.2 Tb/s StrataXGS® Tomahawk® 5 Ethernet Switch Series. <https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm78900-series> [Accessed: Jan 2025].
- [9] Yangtao Deng, Xiang Shi, Zhuo Jiang, Xingjian Zhang, Lei Zhang, Zhang Zhang, Bo Li, Zuquan Song, Hang Zhu, Gaohong Liu, Fuliang Li, Shuguang Wang, Haibin Lin, Jianxi Ye, and Minlan Yu. 2025. Minder: Faulty Machine Detection for Large-scale Distributed Model Training.
- [10] Jianbo Dong, Shaochuang Wang, Fei Feng, Zheng Cao, Heng Pan, Lingbo Tang, Pengcheng Li, Hao Li, Qianyu Ran, Yiqun Guo, Shanyuan Gao, Xin Long, Jie Zhang, Yong Li, Zhisheng Xia, Liuyihan Song, Yingya Zhang, Pan Pan, Guohui Wang, and Xiaowei Jiang. 2021. ACCL: Architecting Highly Scalable Distributed Training Systems With Highly Efficient Collective Communication Library. *IEEE Micro* 41, 5 (2021), 85–92.
- [11] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavam. 2022. Check-N-Run: a Checkpointing System for Training Deep Learning Recommendation Models. In *USENIX NSDI*.
- [12] Swapnil Gandhi, Mark Zhao, Athinagoras Skiadopoulos, and Christos Kozyrakis. 2024. ReCycle: Resilient Training of Large DNNs using Pipeline Adaptation. In *ACM SOSP*.
- [13] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Rifati, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *ACM SIGCOMM*.
- [14] Yanjie Gao, Yu Liu, Hongyu Zhang, Zhengxian Li, Yonghao Zhu, Haoxiang Lin, and Mao Yang. 2020. Estimating GPU Memory Consumption of Deep Learning Models. In *ESEC/FSE*.
- [15] Phillipa Gill, Navendu Jain, and Nachiappan Nagappan. 2011. Understanding Network Failures in Data Centers: Measurement, Analysis, and Implications. In *ACM SIGCOMM*.
- [16] Tao He, Xue Li, Zhibin Wang, Kun Qian, Jingbo Xu, Wenyan Yu, and Jingren Zhou. 2023. Unicorn: Economizing Self-Healing LLM Training at Scale. *arXiv:2401.00134* [cs.DC] <https://arxiv.org/abs/2401.00134>
- [17] Thomas Holterbach, Edgar Costa Molero, Maria Apostolaki, Alberto Dainotti, Stefano Vissicchio, and Laurent Vanbever. 2019. Blink: Fast Connectivity Recovery Entirely in the Data Plane. In *USENIX NSDI*.
- [18] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. 2023. Ooblock: Resilient Distributed Training of Large Models Using Pipeline Templates. In *ACM SOSP*.
- [19] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. In *USENIX ATC*.
- [20] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, et al. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *USENIX NSDI*.
- [21] Raj Joshi, Chahwan Song, Xin Zhe Khooi, Nishant Budhdev, Ayush Mishra, Mun Choon Chan, and Ben Leong. 2023. Masking Corruption Packet Losses in Datacenter Networks with Link-local Retransmission. In *ACM SIGCOMM*.
- [22] Anuj Kalia, Michael Kaminsky, and David G. Andersen. 2016. Design Guidelines for High Performance RDMA Systems. In *USENIX ATC*.
- [23] Baojia Li, Xiaoliang Wang, Jingzhu Wang, Yifan Liu, Yuanyuan Gong, Hao Lu, Weizhen Dang, Weifeng Zhang, Xiaojie Huang, Mingzhuo Chen, Jie Chen, Chunzhi He, Yadong Liu, Xiaoyuan Hu, Chen Liu, Xuefeng Ji, Yinben Xia, Xiang Li, Zekun He, Yachen Wang, and Xian-neng Zou. 2024. TCCL: Co-optimizing Collective Communication and Traffic Routing for GPU-centric Clusters. In *ACM SIGCOMM NAIC Workshop*.
- [24] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *CoRR abs/2006.15704* (2020). *arXiv:2006.15704* <https://arxiv.org/abs/2006.15704>
- [25] Shengkai Lin, Qinwei Yang, Zengyin Yang, Yuchuan Wang, and Shizhen Zhao. 2024. LubeRDMA: A Fail-safe Mechanism of RDMA. In *ACM APNet*.
- [26] Linux Kernel Maintainers. 2018. bonding. <https://wiki.linuxfoundation.org/networking/bonding> [Accessed: Dec 2024].
- [27] Kefei Liu, Jiao Zhang, Zhuo Jiang, Xuan Zhang, Shixian Guo, Yangyang Bai, Yongbin Dong, Zhang Zhang, Xiang Shi, Lei Wang, Haoran Wei, Zicheng Wang, Yongchen Pan, Tian Pan, and Tao Huang. 2024. Hostmesh: Monitor and Diagnose Networks in Rail-optimized RoCE Clusters. In *ACM APNet*.
- [28] AI @ Meta LLAMA Team. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783* [cs.AI] <https://arxiv.org/abs/2407.21783>
- [29] Yuanwei Lu, Guo Chen, Bojie Li, Kun Tan, Yongqiang Xiong, Peng Cheng, Jiansong Zhang, Enhong Chen, and Thomas Moscibroda. 2018. Multi-Path Transport for RDMA in Datacenters. In *USENIX NSDI*.
- [30] Karthik Mandakolathur and Sylvain Jaeger. 2022. Doubling all2all Performance with NVIDIA Collective Communication Library 2.12. <https://developer.nvidia.com/blog/doubling-all2all-performance-with-nvidia-collective-communication-library-2-12/> [Accessed: Jan 2025].
- [31] Microsoft. n.d. Microsoft Collective Communication Library. <https://github.com/Azure/msccl> [Accessed: Jan 2025].
- [32] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. 2021. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *USENIX FAST*.
- [33] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostafa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. *arXiv:2104.04473* [cs.CL] <https://arxiv.org/abs/2104.04473>
- [34] NVIDIA. 2020. Unbreakable Links - MLNX-OS v3.9.0300 - NVIDIA Networking Docs. <https://docs.nvidia.com/networking/display/MLNXOSv390300/Unbreakable+Links>.
- [35] NVIDIA. 2024. Link Aggregation. <https://docs.nvidia.com/docs/sdk/link+aggregation/index.html> [Accessed: Dec 2024].
- [36] NVIDIA. 2024. Network Fabrics — NVIDIA DGX Super-POD. <https://docs.nvidia.com/dgx-superpod/reference-architecture-scalable-infrastructure-h100/latest/network-fabrics.html> [Accessed: Jan 2025].
- [37] NVIDIA. 2024. NVIDIA Resiliency Extension. <https://github.com/NVIDIA/nvidia-resiliency-ext> [Accessed: Dec 2024].

- [38] NVIDIA. n.d. NCCL Tests. <https://github.com/NVIDIA/nccl-tests> [Accessed: Jan 2025].
- [39] NVIDIA. n.d. NVIDIA Collective Communication Library (NCCL) Documentation. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/index.html> [Accessed: August 2024].
- [40] NVIDIA. n.d. Optimized primitives for inter-GPU communication. <https://github.com/NVIDIA/nccl> [Accessed: Jan 2025].
- [41] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *ACM SIGCOMM*.
- [42] Ting Qu, Raj Joshi, Mun Choon Chan, Ben Leong, Deke Guo, and Zhong Liu. 2019. SQR: In-network Packet Loss Recovery from Link Failures for Highly Reliable Datacenter Networks. In *IEEE ICNP*.
- [43] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *ACM KDD*.
- [44] Zhenghang Ren, Yuxuan Li, Zilong Wang, Xinyang Huang, Wenxue Li, Kaiqiang Xu, Xudong Liao, Yijun Sun, Bowen Liu, Han Tian, et al. 2025. Enabling Efficient GPU Communication over Multiple NICs with FuseLink. In *USENIX OSDI*.
- [45] Teven Le Scao et al. 2023. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. arXiv:2211.05100 [cs.CL] <https://arxiv.org/abs/2211.05100>
- [46] Alexander Sergeev and Mike Del Balso. 2018. Horovod: fast and easy distributed deep learning in TensorFlow. *CoRR* abs/1802.05799 (2018). arXiv:1802.05799 <http://arxiv.org/abs/1802.05799>
- [47] Rachee Singh, Muqet Mukhtar, Ashay Krishna, Aniruddha Parkhi, Jitendra Padhye, and David Maltz. 2021. Surviving switch failures in cloud datacenters. *SIGCOMM Comput. Commun. Rev.* 51, 2 (2021), 2–9.
- [48] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *USENIX NSDI*.
- [49] Feng Tian, Wendi Feng, Yang Zhang, and Zhi-Li Zhang. 2020. A Novel Software-based Multi-path RDMA Solution for Data Center Networks. *CoRR* abs/2009.00243 (2020).
- [50] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyan Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rishi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL] <https://arxiv.org/abs/2307.09288>
- [51] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, Ying Zhang, and Naader Hasani. 2024. Rail-only: A Low-Cost High-Performance Network for Training LLMs with Trillion Parameters. In *IEEE HOTI*.
- [52] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. 2023. GEMINI: Fast Failure Recovery in Distributed Training with In-Memory Checkpoints. In *ACM SOSR*.
- [53] Tong Xiao and Jingbo Zhu. 2025. Foundations of Large Language Models. arXiv:2501.09223 [cs.CL] <https://arxiv.org/abs/2501.09223>
- [54] Chenyu Yang, Yuntao Chen, Hao Tian, Chenxin Tao, Xizhou Zhu, Zhaoxiang Zhang, Gao Huang, Hongyang Li, Yu Qiao, Lewei Lu, Jie Zhou, and Jifeng Dai. 2022. BEVFormer v2: Adapting Modern Image Backbones to Bird's-Eye-View Recognition via Perspective Supervision. arXiv:2211.10439 [cs.CV] <https://arxiv.org/abs/2211.10439>
- [55] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2025. A Survey of Large Language Models. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- [56] Danyang Zhuo, Monia Ghobadi, Ratul Mahajan, Klaus-Tycho Förster, Arvind Krishnamurthy, and Thomas Anderson. 2017. Understanding and Mitigating Packet Corruption in Data Center Networks. In *ACM SIGCOMM*.
- [57] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, Steve Lacy, Hang Wang, Aaron Wisner, Chris Lewis, and Henri Bahini. 2024. Resiliency at Scale: Managing Google's TPUnv4 Machine Learning Supercomputer. In *USENIX NSDI*.